

Almond Eye — сопроводительная документация

Прототип для хакатона ДГП Москвы. По фотографиям со стройплощадки система находит технику, понимает, какой этап работ сейчас идёт, сравнивает его с календарным планом и отвечает: успеваем, отстаём или опережаем, и на сколько дней. Каждый вывод подкреплён снимком с отмеченной техникой.

Прототип: backer-overplant-gills.ngrok-free.dev — логин `user`, пароль `bebro4ka_tuva_build_geisha`

1. Как это работает

- Загрузка.** Пользователь загружает календарный план (XLSX в формате шаблона организатора или CSV) и фото с камер. Дату съёмки указывает при загрузке.
- Поиск техники.** Каждое фото проходит через ансамбль моделей (раздел 4). На выходе — список машин и материалов с рамками и уверенностью.
- Определение этапа.** По сочетанию техники система решает, какой этап на фото: например, «экскаватор + самосвалы, без буровых» — котлован. Правила записаны в `backend/app/rules/stage_signatures.yaml`.
- Сравнение с планом.** Для выбранной даты берутся все фото объекта не позже неё. Первое и последнее наблюдение каждого этапа сравниваются с плановыми датами.
- Результат.** Вердикт по объекту, статусы этапов, диаграмма Ганта план/факт и алерты со ссылками на снимки.

Какие выводы делает система:

Ситуация	Вывод
Этап по плану закончен, но на фото после срока всё ещё виден	Отстаём на N дней
Этап по плану должен идти, а видно только предыдущий	Отстаём: этап не начал
Этап виден раньше планового начала	Опережаем на N дней
Этап идёт, но не хватает техники, от которой зависит темп	Риск снижения темпа
На фото техника, которая не нужна ни одному текущему этапу	Техника не по этапу
Нет снимков за последние 45 дней	Нет данных

2. Модель данных

Хранилище — SQLite по умолчанию или PostgreSQL (переменная `DATABASE_URL`). Используются две таблицы.

`projects` — объекты строительства: `id`, `name`, `timezone`.

`documents` — все остальные сущности в виде JSON-документов: `id`, `project_id`, `kind` (тип), `payload` (содержимое), `fingerprint` (SHA-256 содержимого), `created_at`. Документы не изменяются: при повторном сохранении того же содержимого возвращается существующая запись. Благодаря этому сохраняется история: видно, каким планом и какой версией моделей получен каждый вывод.

Тип документа (<i>kind</i>)	Что хранит
<code>camera</code>	камера объекта
<code>zone</code>	зона работ на кадре (полигон в долях кадра)
<code>plan_preview</code> → <code>plan</code>	календарный план: этапы с датами начала и окончания, ключ этапа методики, SHA-256 исходного файла, предупреждения импорта. Сначала создаётся предпросмотр, после подтверждения — сам план
<code>image</code>	снимок: SHA-256, дата съёмки и её источник, размер, флаг качества
<code>image_analysis</code>	результат анализа снимка: найденная техника и материалы с рамками, уверенностью и моделями-источниками; ранжированный список этапов с объяснением; версия моделей и методики
<code>job</code>	фоновая задача анализа пачки снимков: сколько сделано, ошибки, статус
<code>evaluation</code>	покадровая проверка правил (нужная / лишняя техника в зоне)

Сами снимки лежат в `runtime/images/<sha256>`: имя файла — хэш содержимого, поэтому одинаковые фото хранятся один раз.

Справочники заданы в коде и конфигурации:

- типы техники и материалов — `backend/app/domain/contracts.py` (перечисления `Equipment`, `Material`);
- этапы работ и правила «этап ← техника» — `backend/app/rules/stage_signatures.yaml`;
- состав моделей, пороги, проверенные классы — `backend/app/config/detectors.yaml`.

Отклонения не хранятся отдельной таблицей: они вычисляются на лету по плану и анализам снимков (`GET /api/projects/{id}/status?date=`). Так вывод всегда соответствует текущим данным, а историю можно восстановить на любую дату.

3. Код

<code>backend/app/</code>	
<code>main.py</code>	REST API (FastAPI)
<code>storage.py</code>	хранилище (SQLAlchemy)
<code>domain/contracts.py</code>	типы данных: техника, материалы, детекции, этапы
<code>domain/stage_inference.py</code>	техника на фото → этап
<code>domain/progress.py</code>	план + наблюдения → статусы этапов и вердикт
<code>services/detectors/</code>	модели: yolo, grounding (DINO), clip_verify, ensemble, pipeline
<code>services/gantt_import.py</code>	импорт плана XLSX / CSV
<code>config/detectors.yaml</code>	состав ансамбля и пороги
<code>rules/stage_signatures.yaml</code>	методика «этап ← техника»
<code>apps/web/</code>	веб-интерфейс (HTML/JS без сборки)
<code>scripts/</code>	данные, модели, обучение, оценка качества, демо
<code>tests/</code>	105 тестов Python + 18 тестов интерфейса

Предметная логика (`domain/`) написана чистыми функциями без обращения к базе и моделям, поэтому покрыта тестами без GPU. Модели подключаются через общий интерфейс адаптера: новую модель можно добавить строкой в `detectors.yaml`, не меняя остальной код. Комментарии поставлены там, где логика неочевидна: слияние детекций разных моделей, калибровка уверенности, правила вердикта.

Основные эндпоинты API (полное описание — `/docs` на работающем сервере):

Запрос	Назначение
POST /api/projects	создать объект
POST /api/projects/{id}/plans/xlsx/preview, .../confirm	загрузить календарный план
POST /api/projects/{id}/images/batch	загрузить снимки с датой съёмки
POST /api/projects/{id}/analyze-all, GET .../jobs/{job}	запустить анализ и следить за ним
GET /api/projects/{id}/status?date=	вердикт, статусы этапов, алерты
GET /api/projects/{id}/images/{img}/analysis	техника и этапы на конкретном снимке
GET /api/health	состояние моделей

4. Условия и ограничения

Модели и данные, на которых они обучены

Модель	Что находит	Данные / лицензия
SiteSense YOLO26-L	экскаватор, самосвал, бульдозер, погрузчик, автокран, башенный кран, каток, бетономеситель	обучена автором на открытых строительных датасетах; MIT
Grounding DINO base + CLIP ViT-L/14	буровая / сваебойная установка, гусеничный кран, бетононасос, кран-манипулятор; арматура, опалубка, леса, ж/б плиты	без дообучения (поиск по текстовому описанию, проверка кропа); Apache-2.0 / MIT
YOLO11s-seg	бетон, сталь, профнастил (доля в кадре)	дообучена нами на VT Structural Material (3817 фото мостов); CC0
YOLO11s	открытая арматура	дообучена нами на ConRebSeg (2216 кадров со стройки); CC BY 4.0
Construction Hazard YOLO11n	люди, каски, жилеты	только для обзора сцены; AGPL-3.0

Качество (100 фото организатора, ручная разметка)

Пороги подбирались на половине кадров, качество мерилось на другой половине.

Класс	Точность / полнота
экскаватор	0.89 / 0.97
буровая установка	0.91 / 0.91
башенный кран	0.75 / 0.92
гусеничный кран	0.80 / 0.67
погрузчик	0.71 / 0.71
самосвал	0.67 / 0.67 (мало примеров)
автокран	0.36 / 0.50

Этап по фото определяется верно в **71%** кадров (потолок методики при идеальном распознавании техники — 74%). Скорость — около 3 секунд на снимок на RTX 3060.

Для каких этапов настроена логика

Подготовка территории, шпунт и сваи, разработка котлована, фундаменты, подземная часть, установка и демонтаж башенного крана, надземная часть, фасад и кровля, наружные сети, благоустройство. Внутренние работы и ввод объекта по фото не оцениваются: снаружи они не видны.

Лучше всего различаются сваи и котлован. Слабее — фундамент и подземная часть: техника там та же, что на котловане, и различить их можно только по материалам.

Ограничения

- Присутствие техники ещё не означает, что работа идёт. Система оценивает признаки этапа, а не объёмы.
- Алерт «нет нужной техники» выдаётся только для классов с подтверждённой полнотой: экскаватор, буровая, башенный кран. Самосвалы пока распознаются ненадёжно (8 примеров в наборе), поэтому «экскаватор без самосвалов» на реальных детекторах не выдаётся.
- Разметку 100 фото делал один человек, второй проверки не было.
- Календарные планы демо-площадок составлены нами, реальных графиков этих объектов нет.
- Модель материалов обучена на фото мостов и на стройплощадках почти не срабатывает. Она оставлена как вспомогательная.

5. Установка и запуск

Нужны Linux (или Docker), Python 3.12 и [uv](#). GPU NVIDIA желателен, но без него всё тоже работает, только медленнее.

Через Docker:

```
UID=$(id -u) GID=$(id -g) docker compose up -d --build
```

Без Docker:

```
uv sync --extra experiment --group dev
export UV_NO_SYNC=1
```

Данные организатора (архив с фото и перечнем работ) в репозиторий не входят. Их нужно разложить скриптом; он сверит каждый файл по контрольной сумме:

```
uv run scripts/setup_data.py --source <папка или zip от организатора>
```

Модели (~2.7 ГБ, ревизии закреплены):

```
uv run scripts/download_models.py
uv run scripts/download_hazard_model.py
```

Запуск сервера:

```
DETECTOR_PROFILE=ensemble TORCH_DEVICE=0 uv run uvicorn backend.app.main:app --port 7860
```

Без GPU вместо `TORCH_DEVICE=0` укажите `TORCH_DEVICE=cpu`. Интерфейс откроется на <http://localhost:7860>.

Демо-данные — пять площадок с планами и фото:

```
uv run scripts/seed_demo.py
```

Проверка:

```
uv run pytest -q
uv run scripts/eval_detectors.py
uv run scripts/eval_stage_inference.py
```

Дообучение моделей материалов и арматуры не обязательно: без них система работает. Команды для воспроизведения — в [README.md](#), раздел 4.2.

6. Как пользоваться

- 1 На главном экране — все объекты с цветным статусом: «Отстаём на 34 дня», «В графике» и т.д. Для демо откройте «Площадку В».
- 2 **Новый объект** создаётся в три шага: название → календарный план (перетащите XLSX; есть кнопка «Скачать пример плана») → снимки с камер и дата съёмки. Анализ запускается сам, затем открывается статус объекта.
- 3 На странице объекта: вердикт, сводка по этапам, «Что требует внимания» с фото-доказательствами, диаграмма «План и факт» и таблица этапов. Клик по точке на диаграмме или по миниатюре открывает снимок с найденной техникой.
- 4 Кнопки «+ Снимки» и «Обновить план» добавляют новые данные; статус пересчитывается автоматически. Поле «Статус на дату» показывает, как выглядел объект в прошлом.
- 5 «Расширенные настройки» внизу меню — инженерная консоль (зоны на кадре, перечень работ ЛТЦ, CSV-графики). Для обычной работы она не нужна.